

HTTP

PYQ: Define HTTP.

HTTP stands for **HyperText Transfer Protocol**.

It is the **rules** that decide *how data is sent* between a **web browser** (client) and a **web server**.

When you open a website:

- Your browser sends an **HTTP Request** to the server
- The server sends an **HTTP Response** back

HTTP is **stateless**, meaning the server does not remember you between two requests unless other mechanisms (cookies, sessions) are used.

Why we need HTTP?

- To load webpages
- To send form data
- To download/upload files
- To communicate between client and server

Request and Response Message Format

PYQ: Explain about request and response message format with example.

When a browser communicates with a server, they exchange messages in a specific format.

1. HTTP Request Format

A request has 3 main parts:

A) Request Line

Contains:

- **Method** (GET, POST)
- **URL**
- **HTTP Version**

Example:

GET /home HTTP/1.1

B) Headers

Extra information such as:

- Browser type
- Supported languages
- Authentication
- Content type

Example:

Host: www.example.com

User-Agent: Chrome

C) Body (Optional)

Used mainly in **POST**, **PUT**, **PATCH** requests.

Carries data like form inputs, JSON, files.

Example:

username=aashish&password=123

2. HTTP Response Format

A response also has 3 parts:

A) Status Line

Contains:

- HTTP version
- Status code
- Status message

Example:

HTTP/1.1 200 OK

B) Headers

Information sent by server:

Content-Type: text/html

Content-Length: 1200

C) Body

The actual content sent back — HTML, JSON, images, etc.

Example:

```
<html>
<body>Welcome!</body>
</html>
```

Small Example

Client → Server (Request):

GET /about HTTP/1.1

Host: www.example.com

User-Agent: Chrome

Server → Client (Response):

HTTP/1.1 200 OK

Content-Type: text/html

```
<html><body>About Page</body></html>
```

MVC Pattern in ASP.NET Core

Introduction

The **MVC (Model–View–Controller)** pattern is a software design pattern used to build structured and maintainable web applications. It divides an application into **three separate layers**:

- **Model** – Handles data, business rules, and validation.
- **View** – Handles the user interface (HTML generated using Razor).
- **Controller** – Handles user requests, interacts with the Model, and returns the View or data.

This separation keeps code clean, organized, and easy to modify.

How MVC Works – Example of Browsing a Website

Imagine you open your browser and type www.bookstore.com/books to see a list of books.

1. User sends a request

You type the URL **/books** and press Enter.

This sends a request to the web server.

2. Controller receives the request

ASP.NET Core looks at the URL and routes it to the **BooksController**.

Example: BooksController.List() is called.

3. Controller asks the Model for data

The controller says:

"Model, give me the list of books."

So it calls the **BookModel** or BookService to fetch data from a database.

4. Model returns the data

The Model gets the book list (e.g., 50 books) and sends it back to the controller.

5. Controller selects a View

The controller chooses a View file, for example:

Views/Books/List.cshtml

It gives the View the book list received from the Model.

6. View creates HTML and sends it to the browser

The View converts the book data into HTML (a nice table or cards).

The browser finally shows:

"List of Books"

(Book 1, Book 2, Book 3...)

This flow makes MVC a **patterns-based** method for building dynamic websites.

MVC in ASP.NET Core

ASP.NET Core implements MVC as a core framework feature. It provides:

- **Controllers and Actions** to manage requests
- **Models** to represent data and validation
- **Views with Razor syntax** to create dynamic HTML
- **Routing** to map URLs to controllers
- **Tag Helpers** for cleaner HTML
- **Built-in validation** using data annotations
- **Dependency Injection** for cleaner architecture

Because of these features, ASP.NET Core MVC offers a **fast, flexible, and testable** development experience.

Why MVC is Important for Web Applications

- **Separation of concerns** → UI, logic, and data are independent
- **Easy maintenance and scalability**
- **Better testing** due to clear structure
- **Reusable code** → same Model can serve multiple Views
- **Cleaner and organized project structure**
- **Supports both HTML views and APIs**

ASP.NET Core MVC is preferred for modern dynamic web applications because it enforces clean architecture and long-term maintainability.

ASP.NET Core Architecture Overview

ASP.NET Core is built using a **modular, lightweight, and cross-platform architecture**. Its pipeline is highly configurable, making web applications faster and more flexible.

The major building blocks are:

1. Host

ASP.NET Core apps start with a **Generic Host**.

It handles configuration, logging, DI, and the app's lifetime.

2. Middleware Pipeline

Requests pass through a **series of middleware**.

Each middleware can process the request and pass it forward.

Examples: Routing, Authentication, Static Files.

3. Routing

Maps URLs to controllers, Razor pages, or API handlers.

4. MVC Layer

- **Controller:** Receives request
 - **Model:** Handles data/logic
 - **View:** Generates HTML or response
- MVC is optional, but commonly used.

5. Dependency Injection (Built-in)

DI is a **way to provide objects/services to classes instead of creating them inside the class**.

Services like DbContext, logging, or custom classes are registered and injected automatically.

6. Cross-Platform & Web Servers

Runs on Windows, Linux, macOS.

Uses **Kestrel** as the internal server; IIS/Nginx can act as reverse proxies.

Difference From Traditional ASP.NET

- Core is **cross-platform**; old ASP.NET is Windows-only.
- Core has **middleware pipeline**; old ASP.NET has a fixed pipeline.
- Core includes **built-in DI**; old ASP.NET does not.
- Core uses **JSON configuration**; old ASP.NET uses Web.config XML.
- Core uses **Kestrel**; old ASP.NET mainly uses IIS.
- Core is **modular and lightweight**, with higher performance.

Setting Up the ASP.NET Core Environment

1. wwwroot Folder – Your Web Root

- This folder is like the **front of your website**. All static files like CSS, JavaScript, and images go here.
- You can organize files into separate folders like css, js, images.
- Files are accessed through a URL:
<http://localhost:5000/css/app.css>
- You can rename wwwroot to any name and set it as the web root while hosting.

2. Startup Class – The App Manager

- Every ASP.NET Core app must have a **Startup class**, which runs when the app starts.
- It is linked in the Main() method of Program.cs.
It has two main jobs:
 - a) ConfigureServices() – Register Dependencies**
- Think of it as **hiring staff** for your app.
- Here, you register services, classes, or components with the built-in **IoC (Inversion of Control) container**.

- Once registered, these services can be **injected anywhere in the app**.
 - Example: A StudentService class can be used in multiple controllers without creating a new instance every time.

b) Configure() – Set Up Request Pipeline

- Think of this as **arranging the store layout**.
- This method defines **how requests are handled** using IApplicationBuilder.
- Here, you can add **middleware** to:
 - Serve static files from wwwroot
 - Handle routing
 - Handle error

Controllers in ASP.NET Core MVC

- A **Controller** is a class that **handles incoming user requests**, interacts with **Models** to process data, and selects **Views** to display the result.
- Acts as a **bridge between Models and Views**.

How to Create a Controller

Steps:

1. Add a new **class** in the Controllers folder.
2. Name it with the suffix Controller (e.g., StudentController).
3. Inherit from the Controller base class.
4. Create **Action Methods** inside the class.

Example:

```
public class StudentController : Controller
{
    public IActionResult List()
    {
        var students = new List<string> { "Alice", "Bob", "Charlie" };
        return View(students); // sends data to Razor view
    }
}
```

Action Methods

- **Action** → a method in a controller that responds to a user request.
- Action executes logic and returns a result (like HTML, JSON, file, etc.).

Action Result Types

Result Type	What it Does	Example Use
View()	Returns HTML view	Display student list on browser
Json()	Returns JSON data	Send product details to a client app
Redirect()	Redirects to another URL	Move user to login page
Content()	Returns plain text	Send a simple message
File()	Returns a file	Allow download of PDF or image

Common syntax of an Action Method in ASP.NET Core MVC:

```
// Inside a Controller class
public IActionResult ActionName()
{
}
```

```
// Your logic here (like fetching data from Model)

return View(); // or return Json(data), Content("text"), Redirect("url"), etc.
}
```

Example with Multiple Action Types:

```
public class StudentController : Controller
{
    // Returns HTML view
    public IActionResult List()
    {
        var students = new List<string> { "Alice", "Bob", "Charlie" };
        return View(students);
    }
    // Returns JSON data
    public IActionResult GetStudentsJson()
    {
        var students = new List<string> { "Alice", "Bob", "Charlie" };
        return Json(students);
    }
    // Redirect to another action
    public IActionResult GoHome()
    {
        return Redirect("/Home/Index");
    }
    // Return simple text
    public IActionResult Hello()
    {
        return Content("Hello, this is ASP.NET Core MVC!");
    }
}
```

Flow of a Request

1. User types a URL or clicks a link.
2. ASP.NET Core **routing** finds the corresponding controller and action.
3. Action executes logic (may fetch data from model).
4. Action returns a **result** (View, JSON, Redirect, etc.).
5. Browser receives response and displays it.

Rendering HTML with Views (Razor Syntax)

What it is:

Razor is a simple syntax used in ASP.NET Core to combine **HTML and C# code** in the same file. It helps render dynamic content in web pages.

How it works (flow):

1. **Controller Action is called:**
 - User requests a page via URL.
 - The controller receives the request and executes the corresponding action.
2. **Prepare Data (Model):**
 - The action fetches data from a database or creates objects to send to the view.
3. **Pass Data to View:**
 - The action returns a View() and passes the data (Model) to it.
4. **View Renders HTML using Razor:**

- Razor syntax allows embedding C# code inside HTML using @.

5. HTML Sent to Browser:

- The Razor view generates HTML, which the browser displays as a web page.

Requirements for Rendering HTML

- A **Controller** with an Action method that returns a View().
- A **View file** (cshtml) placed under the correct folder (Views/ControllerName/ActionName.cshtml).
- Razor syntax @ to embed dynamic content.

Simple Example

Scenario: Display a list of students.

Controller Action:

```
public class StudentController : Controller
{
    public IActionResult List()
    {
        var students = new List<string> { "Alice", "Bob", "Charlie" };
        return View(students); // Pass data to View
    }
}
```

View (List.cshtml):

```
<h2>Student List</h2>
<ul>
@foreach(var student in Model)
{
    <li>@student</li>
}
</ul>
```

Flow Explanation:

- User goes to /Student/List → controller action List() is called.
- Data (students list) is passed to the view.
- Razor generates HTML: AliceBobCharlie
- Browser displays the list of students.

Understanding Tag Helpers

Tag Helpers are special features in ASP.NET Core that let you use server-side C# code **inside HTML tags**.

They look like normal HTML but are powered by .NET, making your views cleaner and more readable.

Tag Helpers look like normal HTML tags, but they include **special attributes** that start with **asp-***.

Example:

```
<form asp-action="Login"></form>
```

Here, asp-action is a Tag Helper.

Attributes follow this pattern:

```
asp-<feature-name>="value"
```

What Tag Helpers Provide

Tag Helpers provide several benefits:

1. **Cleaner and more natural HTML**
 - Instead of writing confusing Razor code, you use familiar HTML tags with extra attributes.
2. **IntelliSense support in Visual Studio**
 - Editors give suggestions and autocomplete options, making view writing easier.
3. **Consistency and fewer errors**
 - Because Tag Helpers follow MVC conventions, the chances of mistakes reduce.
4. **Server-side processing**
 - Tag Helpers run at the server and generate the final HTML output sent to the browser.
5. **Easy integration with forms, links, validation, environment settings, cache, etc.**
 - ASP.NET Core includes built-in Tag Helpers like Form Tag Helper, Input Tag Helper, Anchor Tag Helper, etc.

Models, Model Binding, and Validation

What is a Model?

A **Model** in ASP.NET Core MVC represents the **data** and the **business rules** of an application.

It contains:

- Properties (fields like Name, Email, Age)
- Validation rules (using data annotations)
- Sometimes business logic (calculations, rules)

Models are used to **pass data between Controller, View, and Database**.

Example Model

```
public class Student {  
    public string Name { get; set; }  
    public int Age { get; set; }  
}
```

Model Binding

Model Binding is the feature in ASP.NET Core that **automatically takes data from an incoming request** (form fields, query strings, route values, JSON body) and **fills your Model object** without any manual mapping.

Why Model Binding Exists

Normally, we would have to write code like:

- `model.Name = form["Name"]`
- `model.Age = Convert.ToInt32(form["Age"])`

This is slow and repetitive.

Model Binding removes this work by doing it automatically.

How Model Binding Works

Imagine a user fills a form on a website:

- Name: Sujal
- Age: 20

When they click "Submit":

1. The browser sends this data to the server.
2. ASP.NET Core receives the data.
3. Your controller action receives a parameter like:
`public IActionResult Register(Student s)`
4. ASP.NET Core looks at the **Student** model:
 - Has a property called **Name**?
 - Has a property called **Age**?
5. ASP.NET Core **automatically fills** the object:
 - `s.Name = "Sujal"`
 - `s.Age = 20`
6. Your action now receives a fully populated object **without writing any mapping code**.

Simple Example Model

```
public class Student {
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Controller

```
[HttpPost]
public IActionResult Register(Student s)
{
    return Content($"Welcome {s.Name}, Age: {s.Age}");
}
```

Here, the browser sends "Name=Sujal&Age=20", and model binding fills the Student s object automatically.

Server-Side Validation

Server-side validation means **the server checks whether the data sent by the user is correct or not** before saving it or using it.

This happens **inside the controller**, after Model Binding.

ASP.NET Core uses **Data Annotations** (attributes written on model properties) to perform these validations automatically.

Why Server-Side Validation Is Needed

- Users can make mistakes (leave field empty, type text in number field, enter invalid email)
- Hackers can bypass client-side validation (disable JavaScript)
- Server must never trust user input
- Prevents wrong data from entering the database

Without server-side validation, your system becomes unreliable and unsafe.

How It Works

Imagine a student fills a form:

- Name: ""
- Email: "notanemail"
- Age: "abc"

User clicks **Submit**.

Here's what happens:

1. Browser sends the data to the server.
2. Model Binding tries to fill your Model object.

3. ASP.NET Core sees the **validation attributes** on your model.
4. It checks whether the data matches those rules.
5. If anything is wrong → ModelState becomes **invalid**.
6. The controller returns the form again with error messages.
7. If everything is correct → The controller continues processing (save to DB, etc.).

The server decides what is valid—not the browser.

Common Validation Attributes (Data Annotations)

Attribute	Meaning
[Required]	Field cannot be empty
[StringLength(50)]	Maximum allowed characters
[Range(1,100)]	Value must fall in range
[EmailAddress]	Must be a valid email
[RegularExpression()]	Custom pattern
[Compare("Password")]	Must match another field

Simple Example Model

```
public class Student
{
    [Required]
    public string Name { get; set; }
    [Range(16, 60)]
    public int Age { get; set; }
    [EmailAddress]
    public string Email { get; set; }
}
```

Controller

```
[HttpPost]
public IActionResult Register(Student s)
{
    if (!ModelState.IsValid)
        return View(s); // return form with validation errors
    return Content("Registration Successful");
}
```

What Happens Internally

1. Data enters the controller.
2. Framework checks all attributes on the model.
3. If any rule fails → ModelState.IsValid becomes **false**.
4. Validation messages are automatically added.
5. The controller returns the same form with errors.
6. If valid → the controller proceeds.

URL Routing and Features

URL Routing in ASP.NET Core is a way to define how the URLs in a web application map to controllers and actions. Instead of relying on physical file paths, routing lets the application decide which code

to execute based on the URL requested by the user.

Think of it like giving each page or action in your site a “nickname” (route), so users can type or click on a friendly URL, and the right controller and action handle it.

How It Works

1. A user types a URL like <https://mysite.com/students/details/5> in the browser.
2. The routing system looks at the URL pattern you’ve defined.
3. It matches the URL to the correct controller (StudentsController) and action method (Details).
4. The action method runs and usually returns a View with the requested data (for student with ID 5).

Features of URL Routing:

- **Custom URLs:** You can define friendly URLs, like /products/view/10 instead of /Products/ViewProduct?id=10.
- **Parameter Handling:** Pass values in the URL directly to your action methods (id, category, etc.).
- **Attribute Routing:** Assign routes directly on controllers or actions using [Route] attributes for more control.
- **Centralized Routing:** Define default patterns in Program.cs or Startup.cs so all URLs follow a consistent structure.

API Controllers

API Controllers in ASP.NET Core are special controllers that handle HTTP requests and send responses in **data format**, usually JSON, instead of HTML. They are mainly used for building **Web APIs** that other applications, like mobile apps or web frontends, can consume.

How API Controllers Work :

Imagine you are building a **Product Management App**. The app needs to:

- Show all products
- Add a new product
- Update a product
- Delete a product

Instead of generating HTML pages like a normal MVC controller, the API controller sends **data** (JSON) to the client. The client (browser, mobile app, or another app) can read and use this data.

Steps in API Controller:

1. **Receive Request:** Client sends a request like GET /products or POST /products.
2. **Controller Processes Request:** The API Controller decides what to do with the request.
3. **Use Models/Services:** Controller can call a service or access the database to get or modify product data.
4. **Return Response:** Controller sends back JSON with product data or confirmation of an action.

Why it’s Useful:

- Data can be used by multiple clients (web, mobile, etc.)
- Easy to work with JSON
- Makes backend separate from frontend, supporting modern apps

Design an API Controller that returns JSON response for a Product entity with CRUD operations.

An API Controller is used to handle **HTTP requests** and **return data**, usually in JSON format, instead of rendering HTML. It's ideal for applications like mobile apps, SPAs (Single Page Applications), or other services that need data.

CRUD

- **Create:** Client sends a new product → API saves it → Responds with success.
- **Read:** Client requests product list → API retrieves products → Responds with JSON list.
- **Update:** Client sends updated product → API updates it → Responds with updated data.
- **Delete:** Client sends delete request → API removes product → Responds with confirmation.

Example Entity:

- Product: ProductId, Name, Price, Category

Example Flow:

1. **Client** (browser/app) sends a request (GET, POST, PUT, DELETE).
2. **API Controller** receives the request.
3. Controller performs **CRUD operation** on the database using the model.
4. Controller returns a **JSON response** back to the client.

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    // GET: api/products
    public IActionResult Get() { ... } // Read all products

    // GET: api/products/1
    public IActionResult Get(int id) { ... } // Read single product

    // POST: api/products
    public IActionResult Post(Product product) { ... } // Create

    // PUT: api/products/1
    public IActionResult Put(int id, Product product) { ... } // Update

    // DELETE: api/products/1
    public IActionResult Delete(int id) { ... } // Delete
}
```

Key Points:

- **Returns JSON** instead of HTML.
- Ideal for **mobile apps, Angular/React frontends, or other services**.
- Each HTTP method corresponds to a **CRUD operation**.

JSON (JavaScript Object Notation)

- **What it is:** A lightweight format to **exchange data** between server and client.
- **Why it's used:** Easy to read for humans and easy to parse for machines.
- **How it works:** API Controllers return data in JSON format instead of HTML.
- **Example:** Product data returned from API:

```
{
  "ProductId": 1,
  "Name": "Laptop",
  "Price": 800,
  "Category": "Electronics"
}
```

- **Use case:** Mobile apps, JavaScript frontends (React, Angular), or any service consuming APIs.

Dependency Injection (DI) and IoC Containers

- **What it is:** A way to provide **required dependencies** (objects/services) to a class instead of creating them inside the class.
- **IoC Container:** Automatically **manages dependencies**, creates objects, and injects them where needed.

Why it's useful:

- Promotes **loose coupling**.
- Makes code easier to **test and maintain**.
- Allows **reusing services** across the application.

Simple Example

- You have a **LibraryService** class that needs a **BookRepository** to fetch data.
- Instead of creating BookRepository inside LibraryService, the IoC container injects it automatically.
- Now, if you change repository implementation later, LibraryService doesn't need to change.

ADO.NET Basics

ADO.NET is a technology in .NET used to communicate with a database. It lets your C# program **connect, send queries, read data, and update tables**.

1. Connection Class

The **Connection** object creates a link between your application and the database.

- It stores **connection string** (server name, database name, security).
- It must be **opened** before sending queries.
- It must be **closed** after work is done.

Syntax

```
SqlConnection conn = new SqlConnection(connectionString);
conn.Open();
conn.Close();
```

Explanation

- SqlConnection creates a link between your C# program and SQL Server.
- Open() activates the connection.
- Close() or using block closes it safely.
- Connection must be opened before running any SQL command.

Short Code Example

```
string connectionString = "Server=.;Database=IOST;Trusted_Connection=True;";
using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();
    Console.WriteLine("Connection Successful!");
}
```

2. Command Class

The **Command** object is used to send instructions to the database.

It can execute:

- **SELECT** (to read data)
- **INSERT**

- **UPDATE**
- **DELETE**
- Stored procedures

You create a command and attach it to a **Connection**.

3. Reader Class

The **DataReader** reads data **one row at a time**, only forward.

Used for **fast reading** of large results.

- It works only with **SELECT** queries.
- It requires the **Connection** to remain open while reading.

4. Adapter, DataSet, DataTable

DataAdapter

- Acts as a **bridge** between the application and the database.
- Uses four commands:
 - **SelectCommand**
 - **InsertCommand**
 - **UpdateCommand**
 - **DeleteCommand**
- Supports **disconnected architecture**:
 - Opens connection when needed
 - Retrieves data
 - Closes connection
- Used to **fill** DataSet/DataTable and **update** changes back to the database.

How it works

1. Opens connection
2. Executes SELECT
3. Loads data into DataSet/DataTable
4. Closes connection
5. Later applies INSERT/UPDATE/DELETE based on changes.

DataSet

- An **in-memory database**.
- Can store **multiple tables**.
- Does **not** directly interact with the database.
- Used when working with multiple related tables or offline data.

DataTable

- A **single in-memory table** inside DataSet.
- Stores data as **rows and columns**.
- Supports operations like:
 - Add row
 - Edit row
 - Delete row
 - Filter/Sort locally
- Changes made here are later saved to the database using DataAdapter.

How They Work Together

1. **DataAdapter** runs SELECT → fills **DataSet**.
2. **DataSet** stores data in a **DataTable**.
3. App works on DataTable **in memory** (add/edit/delete rows).
4. **DataAdapter.Update()** sends all changes back to the real database using commands.

Role of Adapter Class

- Connects DataSet/DataTable with the actual database.
- Executes SELECT, INSERT, UPDATE, DELETE commands.
- Supports disconnected mode by opening and closing connections automatically.
- Updates the actual database based on changes in DataTable.

How They Work Together

1. **Connection** opens the link to the database.
2. **Command** sends the SQL query.
3. If the query is SELECT:
 - **Reader** reads the data row-by-row.
4. If we want a whole table or offline data:
 - **Adapter** fills DataSet and can update back to DB.
5. **Connection** closes after work.

INSERTING DATA

Syntax

```
SqlCommand cmd = new SqlCommand("INSERT INTO Table VALUES (...)", conn);  
cmd.ExecuteNonQuery();
```

Explanation

- SqlCommand sends SQL queries (INSERT/UPDATE/DELETE) to the database.
- ExecuteNonQuery() executes commands that don't return data.
- Runs only when the connection is already open.

Short Code Example

```
string cs = "Server=.;Database=I0ST;Trusted_Connection=True;";  
using (SqlConnection conn = new SqlConnection(cs))  
{  
    conn.Open();  
    string query = "INSERT INTO Faculty (Course_Name, CourseID, No_of_Semester, Fee) " +  
        "VALUES ('CSIT', 'C101', 8, 300000)";  
    SqlCommand cmd = new SqlCommand(query, conn);  
    cmd.ExecuteNonQuery();  
    Console.WriteLine("Record Inserted!");  
}
```

READING DATA USING SQLDATAADAPTER + DATASET

Syntax

```
SqlDataAdapter da = new SqlDataAdapter(query, conn);  
DataSet ds = new DataSet();  
da.Fill(ds, "TableName");
```

Explanation

- SqlDataAdapter fetches data from the database.
- DataSet is an **in-memory container** holding one or more DataTables.
- Fill() loads the result of SELECT query into the DataSet.
- No need to keep the connection open after filling — DataSet works offline.

Short Code Example

```
string cs = "Server=.;Database=IOST;Trusted_Connection=True;";
using (SqlConnection conn = new SqlConnection(cs))
{
    conn.Open();
    string query = "SELECT Course_Name, Fee FROM Faculty WHERE Course_Name = 'CSIT'";
    SqlDataAdapter da = new SqlDataAdapter(query, conn);
    DataSet ds = new DataSet();
    da.Fill(ds, "Faculty");
    foreach (DataRow row in ds.Tables["Faculty"].Rows)
    {
        Console.WriteLine(row["Course_Name"].ToString() + " - " + row["Fee"].ToString());
    }
}
```

ORM

ORM (Object–Relational Mapping) is a technique that connects:

- **Classes ↔ Database Tables**
- **Objects ↔ Rows**

It converts data between the program (objects) and the database (tables).

Why ORM is Useful

Databases use rows and columns.

Programs use objects and properties.

ORM removes the difference and lets you work naturally with objects.

What ORM Does

- Maps class → table
- Maps property → column
- Automatically builds SQL
- Converts table rows into objects
- Simplifies CRUD operations
- Manages relationships

Imagine you have a **Student** class in C#:

Student

- Id
- Name
- Age

An ORM (like EF Core) does this automatically:

Mapping

Student class → Students table

Id → Id column

Name → Name column

Age → Age column

When you save a new student object

You create:

```
Student s = new Student("Aashu", 22)
```

ORM generates SQL internally:

INSERT INTO Students (Name, Age) VALUES ('Aashu', 22)

But you **never write this SQL yourself**.

When you read students

ORM generates:

SELECT * FROM Students

And returns a **list of Student objects**.

So:

- You think in **objects**
- ORM handles the **database**

This is the entire idea behind ORM and EF Core.

Entity Framework (EF) Core

EF Core is a modern **Object-Relational Mapper (ORM)** for .NET.

It allows you to work with a database **using C# classes and objects**, instead of writing SQL queries manually.

EF Core automatically:

- Creates database tables from your classes
- Inserts, updates, deletes data
- Reads data and returns objects
- Handles relationships (One-to-Many, Many-to-Many)
- Generates SQL internally so you don't have to write it

Goal: Make database work simpler and faster for developers.

Key Features of EF Core

1. **Works with objects** – no SQL required for most tasks.
2. **LINQ support** – query database using C# syntax.
3. **Cross-platform** – works on Windows, Linux, macOS.
4. **Migration system** – updates database structure automatically.
5. **Tracks changes** – detects what data changed and updates only that.
6. **Supports multiple databases** – SQL Server, MySQL, SQLite, PostgreSQL, etc.

Advantages of EF Core over ADO.NET

- No need to write long SQL queries for CRUD
- Saves time and reduces errors
- More readable, clean, object-based code
- Handles relationships automatically
- Built-in validation and mapping
- Uses LINQ queries which are type-safe
- Better for large applications with many tables

Adding EF Core to an Application

Before using EF Core, we must **add it to a project** and tell it **which database** we want to connect to.

Steps

1. **Install EF Core packages** (for example: EF Core + SQL Server provider).
2. **Choose database provider**
 - SQL Server
 - MySQL
 - PostgreSQL

- SQLite
- 3. **Configure EF Core** in the project so it knows
 - which database to use
 - how to connect
 - what models it will map
- 4. **Create Data Models and Data Context** (explained below).

Data Models and Data Context

(PYQ: Define data model and data context)

1. Data Model

Easy definition:

A **data model** is a **C# class that represents a table** in the database.

Each **property** → **column** of the table.

Example

If you have a table **Employee(Id, Name, Salary)**,
you create a **model class Employee** with:

- Id
- Name
- Salary

Purpose:

- Defines structure of data
- EF Core uses it to create tables and handle CRUD

2. Data Context

Easy definition:

A **data context** is a **bridge between your C# application and the database**.

It:

- Manages the database connection
- Maps models to tables
- Tracks changes in data
- Sends queries to the database
- Saves updates to the database

Think of it like:

A “database manager” inside your application.

Conceptual example

A context class may contain:

- Employees table
- Products table
- Orders table

EF Core uses the context to know:

- Which tables exist
- How they relate
- How to insert/update/delete data

PYQ :

Create a table BAG(ID, Brand, Price, Date) and perform the following operations using Entity Framework Core:

1. **Update the Price to 2000 for books published in 2022**
2. **Delete books published in 1981**
3. **Retrieve the Brand of books whose price is greater than 500**

1. Create the Model (Data Model)

This represents the **BAG** table.

```
public class Bag
{
    public int ID { get; set; }
    public string Brand { get; set; }
    public int Price { get; set; }
    public int Date { get; set; } // Year of publication
}
```

Explanation

- EF Core will treat this class as a table.
- Each property becomes a column.
- Date is just an integer representing year.

2. Create the Data Context (DbContext)

This class connects the model to the database.

```
public class AppDbContext : DbContext
{
    public DbSet<Bag> Bags { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer("Your_Connection_String_Here");
    }
}
```

Explanation

- DbSet<Bag> means EF Core will create a **Bags** table.
- UseSqlServer tells EF Core which database provider we are using.
- The connection string connects to SQL Server.

3. Perform the Required Operations

We now solve all three tasks given in the question.

A. Update Price to 2000 for books published in 2022

```
using (var db = new AppDbContext())
{
    var items2022 = db.Bags.Where(b => b.Date == 2022).ToList();
    foreach (var item in items2022)
    {
        item.Price = 2000;
    }
    db.SaveChanges();
}
```

Explanation

- Where(b => b.Date == 2022) selects only 2022 records.
- Loop updates Price to 2000.
- SaveChanges() writes the update into the database.

B. Delete books published in 1981

```
using (var db = new AppDbContext())
{
    var oldBooks = db.Bags.Where(b => b.Date == 1981).ToList();
    db.Bags.RemoveRange(oldBooks);
    db.SaveChanges();
}
```

Explanation

- First, EF Core fetches items from 1981.
- RemoveRange deletes all selected rows.
- SaveChanges() applies deletion to DB.

C. Retrieve Brand of books with Price > 500

```
using (var db = new AppDbContext())
{
    var expensiveBrands = db.Bags
        .Where(b => b.Price > 500)
        .Select(b => b.Brand)
        .ToList();
    foreach (var brand in expensiveBrands)
    {
        Console.WriteLine(brand);
    }
}
```

Explanation

- Where(b => b.Price > 500) filters rows.
- Select(b => b.Brand) retrieves only the Brand column.
- Loop prints the result.